Langchian , RAG INTVW QUESTIONS

1. **LangChain prompt utilities and document transformers**
2. **Dynamic tool registration in LLM agents**
3. **Synchronous vs asynchronous API design (FastAPI, LangChain)**
4. **RAG ingestion patterns and zero-downtime indexing**
5. **PDF chunking, provenance, and citation retrieval**
6. **LlamaIndex query lifecycle hooks and callbacks**
7. **Re-ranking, hybrid search, and latency optimization**
8. **LLM batching, concurrency control, and performance tuning**
9. **Metadata filtering and soft deletes in vector indexes**
10. **Retriever explainability and scoring diagnostics**
11. **Prompt engineering for emotional and creative writing**
12. **Reinforcement learning concepts (reward shaping)**
13. **Embeddings for semantic search**
14. **Retrieval-augmented generation for large corpora**
15. **Prompt chaining and context management**
16. **Controlling verbosity via prompts and parameters**
17. **Preserving numerical accuracy in summarization**
18. **Zero-shot vs one-shot prompting**
19. **Fine-tuning bias mitigation**
20. **Role-based prompting for domain specificity**
21. **Prompt design for legal/technical summarization**
22. **Constrained output classification**
23. **Python decorators and memoization**
24. **Python OOP (inheritance, method overriding, super)**
25. **Python multithreading and locks**
26. **Python execution order and nondeterminism**
27. **Algorithmic problems (Fibonacci, prime factors)**

30 Interview Questions (with Answers)

LangChain / LlamaIndex / RAG / LLM Systems

1. **You need to remove fields from retrieved documents at prompt time without touching the vector store. Which LangChain utility fits best?**
   Answer: Filter / Document Transformer
2. **Business users must add tools at runtime without redeploying the agent. Best design?**
   Answer: Store tool definitions in DB and load dynamically at runtime
3. **Expose sync and async APIs for the same chain without duplicating logic. Best approach?**
   Answer: Use the same chain and call/invoke (sync + async)
4. **RAG system must ingest daily docs without downtime. Best ingestion pattern?**
   Answer: Build new index and swap atomically (blue–green index)
5. **Need page-level provenance from PDFs for exact citations. Best approach?**
   Answer: PDF page splitter + metadata
6. **Where to inject hashing/signing of responses without breaking answer flow (LlamaIndex)?**
   Answer: Override `query()` and wrap post-processing in try/except
7. **Hybrid retriever latency spikes due to reranker CPU bursts. Least invasive fix?**
   Answer: Replace reranker with lightweight model in separate thread pool
8. **High P95 latency due to many concurrent LLM calls. Best mitigation using primitives?**
   Answer: Enable batched LLM calls
9. **Stale documents appear despite soft deletes. Fix without reindex?**
   Answer: Enable metadata filtering in retriever
10. **Need explainability of retriever scores with minimal overhead. Best option?**
    Answer: Score diagnostics retriever

Prompt Engineering / LLM Behavior

11. **Prompt for deep emotional conflict (guilt + love). Best choice?**
    Answer: Write a scene explicitly showing both emotions in conflict
12. **Rewarding concise, accurate summaries via feedback is an example of?**
    Answer: Reinforcement learning (reward shaping)
13. **How do embeddings improve search relevance?**
    Answer: Convert queries and documents into vectors for similarity search

14. **Database too large for context window. Best embedding-based solution?**
    Answer: Embeddings + similarity-based retrieval (RAG)
15. **Chained prompts produce generic output. How to fix?**
    Answer: Combine prior responses into a single final prompt
16. **Assistant responses are too verbose. Best prompt fix?**
    Answer: Explicit length constraint (e.g., "respond in <20 words")
17. **Summaries miss critical numbers. How to fix?**
    Answer: Explicitly instruct model to preserve numerical data
18. **Zero-shot classifier outputs invalid labels. Best improvement?**
    Answer: Add a one-shot example
19. **Fine-tuned recommender biased to one category. How to mitigate?**
    Answer: Add balanced and diverse fine-tuning examples
20. **Domain-specific answers without fine-tuning. Best method?**
    Answer: Role-based system prompt

Python / Concurrency / OOP / Algorithms

21. **Memoized Fibonacci called twice with same input. Output?**
    Answer: Same value both times (cached result)
22. **Increment and decrement threads with a lock finish execution. Final value?**
    Answer: 0
23. **Two threads printing numbers 1–5 without synchronization. Output?**
    Answer: Order is nondeterministic
24. **Filtering even numbers from Fibonacci sequence up to 8 terms gives?**
    Answer: [0, 2, 8]
25. **Largest prime factor of 15?**
    Answer: 5
26. **Largest prime factor of 256?**
    Answer: 2
27. **Base class constructor calls method overridden in subclass. Which method runs?**
    Answer: Subclass method (dynamic dispatch)
28. **Using `super().__init__()` ensures what?**
    Answer: Base class initialization executes
29. **Why can threading outputs interleave in Python?**
    Answer: OS-level scheduling + GIL context switching
30. **When is multiprocessing preferred over threading in Python?**
    Answer: CPU-bound workloads

30 Advanced Interview Questions (with Short Explanations)

LLM Systems & Architecture

1. **Why is RAG preferred over fine-tuning for frequently changing data?**
   Answer: RAG decouples knowledge from the model. Updating data requires re-indexing, not retraining, which is faster and cheaper.
2. **What problem does context window limitation create in LLM applications?**
   Answer: Large documents cannot fit in a single prompt, causing information loss unless retrieval or chunking is used.
3. **When would you choose hybrid search over pure vector search?**
   Answer: When exact keyword matching and semantic similarity are both important (e.g., legal or code search).
4. **Why is metadata filtering critical in enterprise RAG systems?**
   Answer: It prevents stale, unauthorized, or irrelevant documents from being retrieved.
5. **What causes hallucinations in RAG pipelines even with retrieval?**
   Answer: Poor chunking, low-quality retrieval, or missing grounding instructions.

LangChain / LlamaIndex / Agent Design

6. **Why should tools be registered dynamically instead of statically?**
   Answer: It allows runtime extensibility without redeployment, critical for business-driven systems.
7. **What is the advantage of callback handlers in LangChain/LlamaIndex?**
   Answer: They allow instrumentation, logging, auditing, and safety logic without modifying core logic.
8. **Why is blue-green indexing safer than in-place updates?**
   Answer: It avoids downtime and prevents partial or corrupted indexes from serving traffic.
9. **Why are rerankers often the performance bottleneck?**
   Answer: They perform pairwise or cross-encoder scoring, which is CPU/GPU expensive.
10. **What is the role of a router chain in agent systems?**
    Answer: It selects the correct tool or sub-chain based on intent, improving accuracy and modularity.

Performance & Scalability

11. **Why does batching reduce LLM latency under high QPS?**
    Answer: It amortizes network and model overhead across multiple requests.
12. **Why is async I/O critical in LLM-backed APIs?**
    Answer: LLM calls are I/O-bound; async prevents thread blocking and improves throughput.
13. **What problem does back-pressure solve in LLM services?**
    Answer: It prevents system overload by limiting concurrent in-flight requests.
14. **Why cache embeddings instead of full responses?**
    Answer: Embeddings are reusable across queries, while responses are query-specific.
15. **When should you offload inference to GPU vs CPU?**
    Answer: GPU for batch or heavy inference; CPU for low-latency or lightweight models.

Prompt Engineering & Alignment

16. **Why are explicit constraints better than temperature tuning for verbosity control?**
    Answer: Constraints directly shape output structure; temperature only affects randomness.
17. **Why do role-based prompts improve domain accuracy?**
    Answer: They prime the model's latent knowledge and reasoning style.
18. **Why does one-shot prompting often outperform zero-shot classification?**
    Answer: It anchors the model to the expected output format.
19. **Why do summaries miss numbers without explicit instruction?**
    Answer: LLMs prioritize semantic meaning over exact values unless instructed otherwise.
20. **Why is prompt chaining risky without state consolidation?**
    Answer: Context fragmentation leads to generic or inconsistent outputs.

Security, Safety & Reliability

21. **Why should signing/logging occur after response generation?**
    Answer: Failures in logging must not block user responses.
22. **What is prompt injection in RAG systems?**
    Answer: Malicious content in retrieved documents overrides system instructions.

23. **How does document-level authorization reduce data leakage?**
    Answer: It enforces access control before retrieval, not after generation.
24. **Why are soft deletes risky without filters?**
    Answer: Deleted content can still be retrieved unless explicitly excluded.
25. **Why should LLM outputs be validated before execution?**
    Answer: To prevent unsafe or malformed tool invocations.

Python / Backend Engineering

26. **Why is threading unreliable for CPU-bound tasks in Python?**
    Answer: The GIL prevents true parallel execution.
27. **Why does method overriding work inside constructors?**
    Answer: Python uses dynamic dispatch, even during initialization.
28. **Why is locking necessary even for simple increments?**
    Answer: Operations are not atomic; race conditions can corrupt state.
29. **Why are decorators useful for caching and instrumentation?**
    Answer: They add cross-cutting concerns without changing core logic.
30. **Why is deterministic behavior important in LLM pipelines?**
    Answer: It ensures reproducibility, debuggability, and audit compliance.